Final Project - Emery Individual Section

Emery Berry

How do quality-of-life detractors (crime rate, air pollution, and lower socioeconomic status) impact median house prices in Boston?

Q1 - Context and Scope of the Problem

The goal of this project is to discover how a specific category of variables will affect median house prices and to use these variables for prediction. The data set used contains 506 observations across 13 predictors capturing various neighborhood characteristics throughout Boston, MA. The response variable of interest is `medv`, the median home value measured in thousands of dollars.

This analysis focuses specifically on **socioeconomic and environmental risk factors**, which are variables that reflect adverse neighborhood conditions rather than physical housing attributes. The three predictors of interest are:

- `lstat` — the percentage of lower status population in the neighborhood, indicating socioeconomic disadvantage
- `crim` — the per capita crime rate by town, reflecting public safety conditions
- `nox` — the concentration of nitrogen oxides in the air (parts per 10 million), serving as a measure of environmental quality and pollution

Each of these variables is expected to have a negative relationship with median home value. It is expected that as crime rises, air quality worsens, or socioeconomic disadvantage increases, home prices tend to fall. However, the nature of these relationships may not be strictly linear. For example, there may be a threshold where home prices drop sharply only after crime or pollution crosses a certain level. Understanding the nature of these relationships, not just their direction, is a key motivation for this analysis. Ultimately, this work aims to answer: **How do socioeconomic and environmental risk factors influence median house prices in Boston, and how well can these variables alone predict home value?**

Why This Analysis Matters This analysis is relevant across multiple domains. For policymakers and urban planners, quantifying how crime and pollution suppress home values

provides evidence to justify targeted investment in underserved neighborhoods. For real estate professionals, these variables can serve as diagnostic tools for more accurate pricing and risk assessment. More broadly, the variables examined here (crime, pollution, and socioeconomic status) disproportionately affect low-income and historically marginalized communities, meaning their impact on home prices reflects deeper structural inequalities baked into the housing market. Understanding these relationships is a step toward making those inequalities visible and, ideally, actionable.

Q2 - Methods and Results

To answer the research question, three statistical learning models were fit to the Boston dataset using `lstat`, `crim`, and `nox` as predictors of median house price. Each model was tuned using cross-validation and compared based on predictive performance. The three models chosen were a Generalized Additive Model (GAM), Random Forest, and Principal Components Regression (PCR). GAM was chosen because it can capture flexible, nonlinear relationships between each predictor and the response. This makes sense in the context of the problem since the effects of crime, pollution, and socioeconomic status on home prices probably aren't strictly linear. Random Forest was included because it handles complex interactions between predictors well without requiring any assumptions about the functional form, and its variable importance output gives us a clearer picture of which risk factors matter most. PCR rounds out the analysis by reducing the three predictors into uncorrelated components, which helps us understand how much of the variation in home prices can be explained by the combined structure of these variables. Together, the three models take quite different approaches to the same problem, which should give us a more complete picture of how these neighborhood characteristics relate to median home value.

Setup

```
library(ISLR2)
data(Boston)

boston_emery <- Boston[, c("crim", "nox", "lstat", "medv")]

head(boston_emery)
```

```
     crim   nox lstat medv
1 0.00632 0.538  4.98 24.0
2 0.02731 0.469  9.14 21.6
3 0.02729 0.469  4.03 34.7
4 0.03237 0.458  2.94 33.4
5 0.06905 0.458  5.33 36.2
```

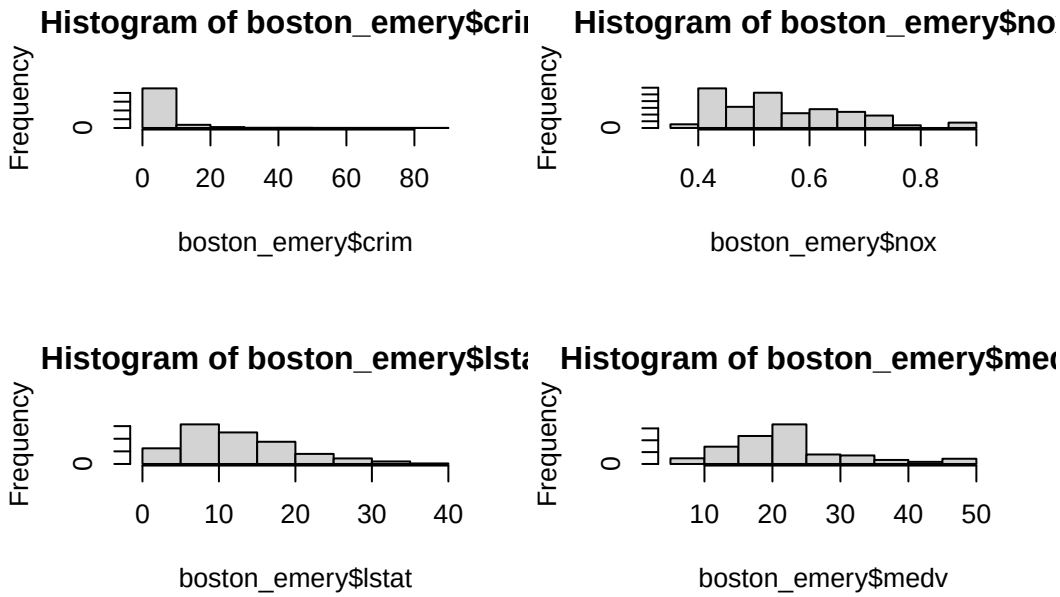
```
6 0.02985 0.458  5.21 28.7
```

```r
summary(boston_emery)
```

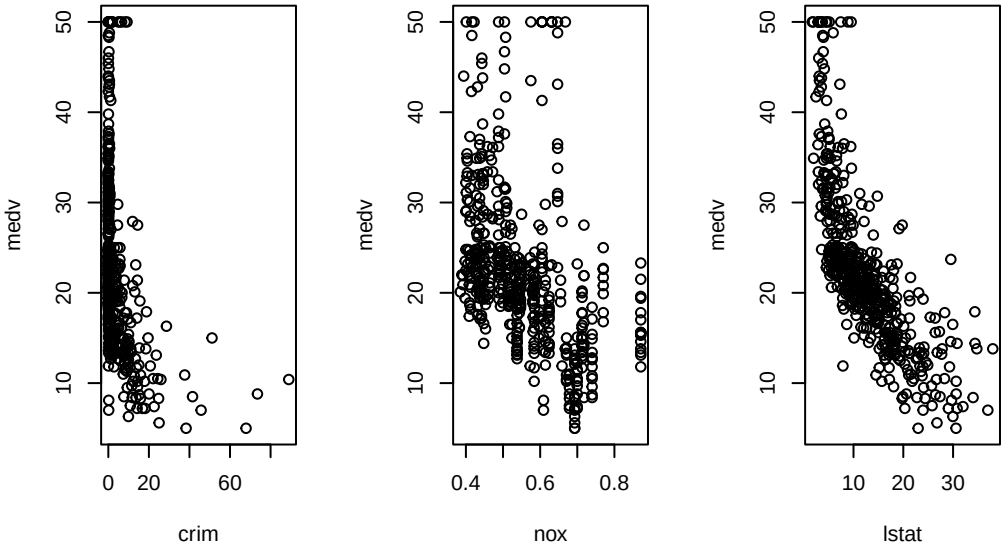
```
      crim                nox              lstat             medv      
 Min.   : 0.00632   Min.   :0.3850   Min.   : 1.73   Min.   : 5.00  
 1st Qu.: 0.08205   1st Qu.:0.4490   1st Qu.: 6.95   1st Qu.:17.02  
 Median : 0.25651   Median :0.5380   Median :11.36   Median :21.20  
 Mean   : 3.61352   Mean   :0.5547   Mean   :12.65   Mean   :22.53  
 3rd Qu.: 3.67708   3rd Qu.:0.6240   3rd Qu.:16.95   3rd Qu.:25.00  
 Max.   :88.97620   Max.   :0.8710   Max.   :37.97   Max.   :50.00  
```

Exploratory Analysis

```r
# Distribution of each variable
par(mfrow = c(2, 2))
hist(boston_emery$crim)
hist(boston_emery$nox)
hist(boston_emery$lstat)
hist(boston_emery$medv)
```



```r
# Scatterplots of each predictor vs response
par(mfrow = c(1, 3))
plot(boston_emery$crim, boston_emery$medv, xlab = "crim", ylab = "medv")
plot(boston_emery$nox, boston_emery$medv, xlab = "nox", ylab = "medv")
plot(boston_emery$lstat, boston_emery$medv, xlab = "lstat", ylab = "medv")
```




```r
cor(boston_emery[, c("crim", "nox", "lstat")])
```

```
           crim       nox     lstat
crim  1.0000000 0.4209717 0.4556215
nox   0.4209717 1.0000000 0.5908789
lstat 0.4556215 0.5908789 1.0000000
```

The exploratory analysis reveals several important characteristics of the data. The `crim` variable is heavily right-skewed with a handful of extreme outliers, `nox` is roughly bimodal, and `lstat` is moderately right-skewed. The scatterplots confirm that all three predictors have a negative relationship with `medv`, though none are strictly linear. `crim` in particular shows a strong downward relationship after a certain threshold and `lstat` shows the most consistent downward trend. The correlation matrix further reveals moderate positive correlations between all three predictors, with `nox` and `lstat` being the most related at 0.59, which makes intuitive sense as higher pollution areas tend to overlap with lower socioeconomic neighborhoods. Taken together, the nonlinear predictor-response relationships justify the use of GAM as one of the models used, the moderate predictor correlations provide motivation for PCR, and the overall complexity of the data suggests that Random Forest's flexibility in capturing interactions will be valuable.

Set Up Training and Testing Data

```
set.seed(1)
n <- nrow(boston_emery)
train_idx <- sample(1:n, size = 0.8 * n)

train <- boston_emery[train_idx, ]
test  <- boston_emery[-train_idx, ]
```

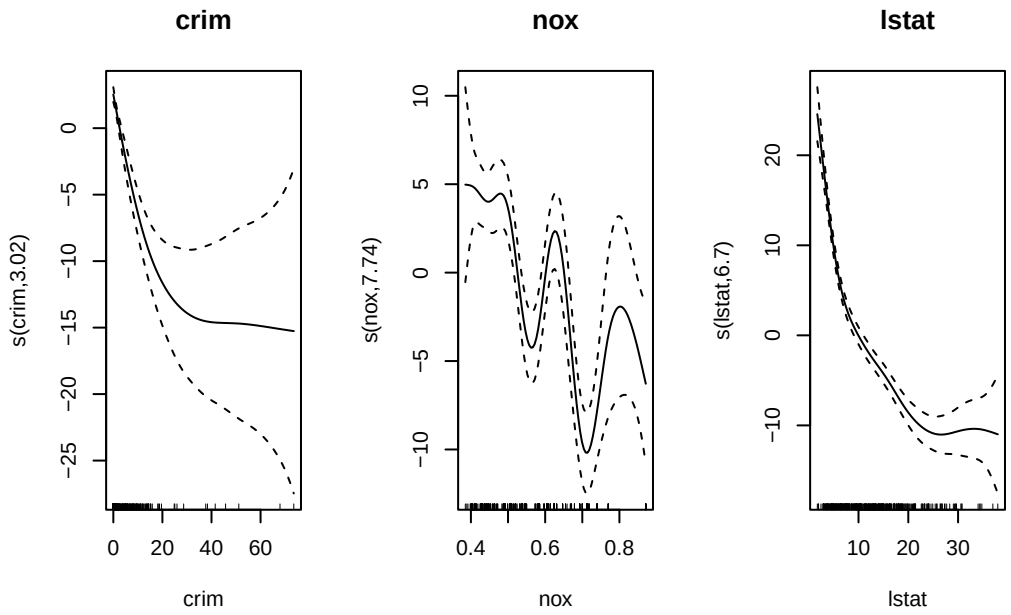
Generalized Additive Model

```
library(mgcv)
```

```
Loading required package: nlme
```

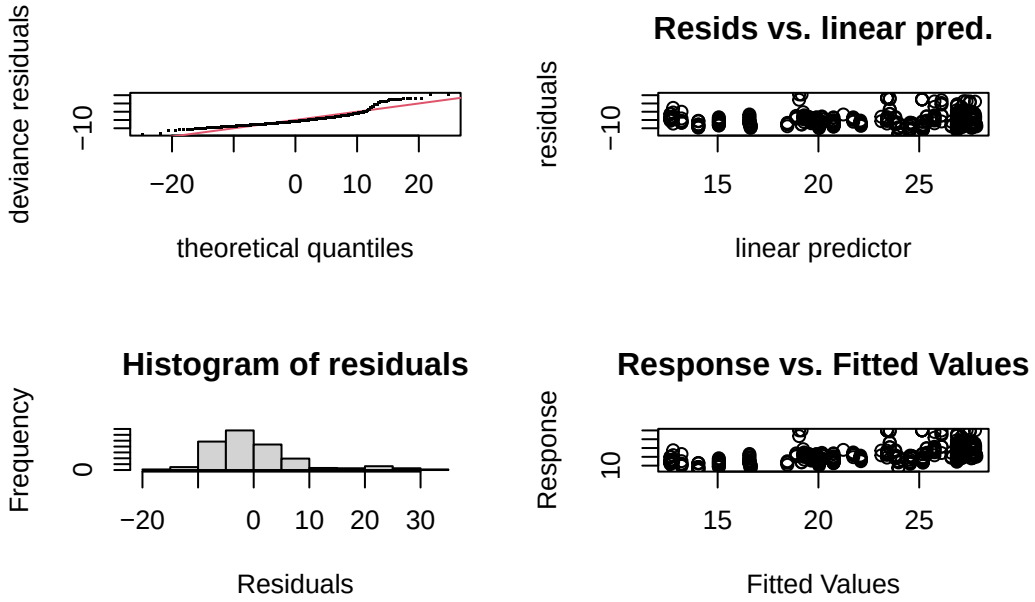
```
This is mgcv 1.9-4. For overview type '?mgcv'.
```

```
# Individual GAMs for each predictor
gam_crim <- gam(medv ~ s(crim), data = train, method = "REML")
gam_nox <- gam(medv ~ s(nox), data = train, method = "REML")
gam_lstat <- gam(medv ~ s(lstat), data = train, method = "REML")

# Plot each smooth term individually
par(mfrow = c(1, 3))
plot(gam_crim, main = "crim")
plot(gam_nox, main = "nox")
plot(gam_lstat, main = "lstat")
```



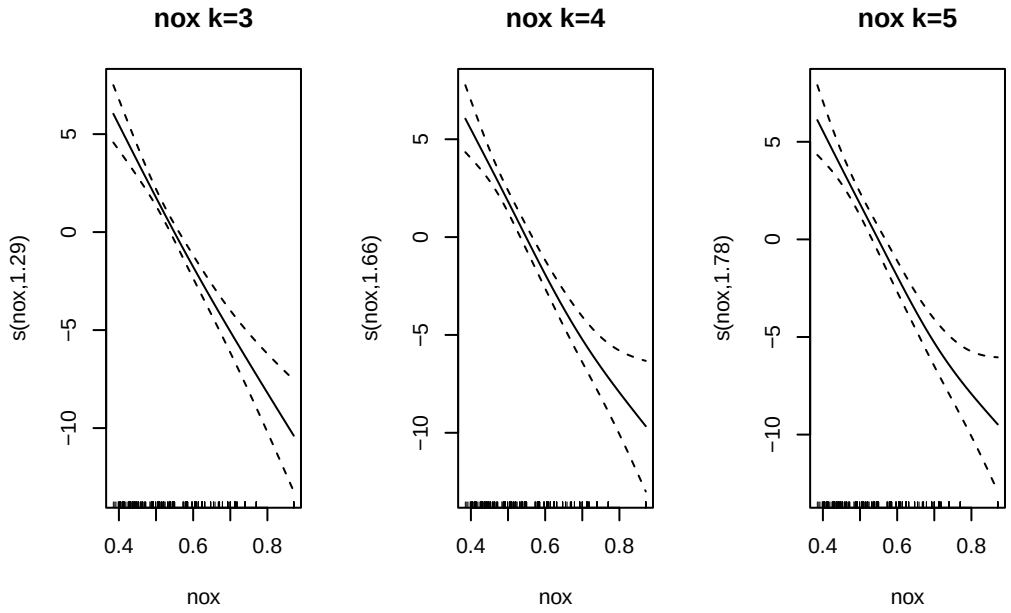
The individual thin plate regression spline plots revealed that `crim` and `lstat` both show clean nonlinear relationships with `medv`, justifying the use of splines for each. However, the `nox` spline appeared overly wiggly with wide confidence intervals and a high edf of 7.74, suggesting the model may be overfitting for this predictor.

```
# Run gam.check on individual nox model
gam.check(gam_nox)
```



```
Method: REML   Optimizer: outer newton
full convergence after 7 iterations.
Gradient range [-0.000170038,3.320706e-07]
(score 1434.431 & scale 67.10458).
Hessian positive definite, eigenvalue range [2.30361,201.0574].
Model rank =  10 / 10

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

         k'  edf k-index p-value
s(nox) 9.00 7.74     0.6  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Refit nox model with lower k values and compare
gam_nox_k3 <- gam(medv ~ s(nox, k = 3), data = train, method = "REML")
gam_nox_k4 <- gam(medv ~ s(nox, k = 4), data = train, method = "REML")
gam_nox_k5 <- gam(medv ~ s(nox, k = 5), data = train, method = "REML")

# Plot each to compare wiggliness
par(mfrow = c(1, 3))
plot(gam_nox_k3, main = "nox k=3")
plot(gam_nox_k4, main = "nox k=4")
plot(gam_nox_k5, main = "nox k=5")
```




```
# Compare AIC
AIC(gam_nox, gam_nox_k3, gam_nox_k4, gam_nox_k5)
```

```
                df      AIC
gam_nox     10.243274 2857.476
gam_nox_k3   3.499539 2893.475
gam_nox_k4   4.033718 2893.472
gam_nox_k5   4.216084 2893.493
```

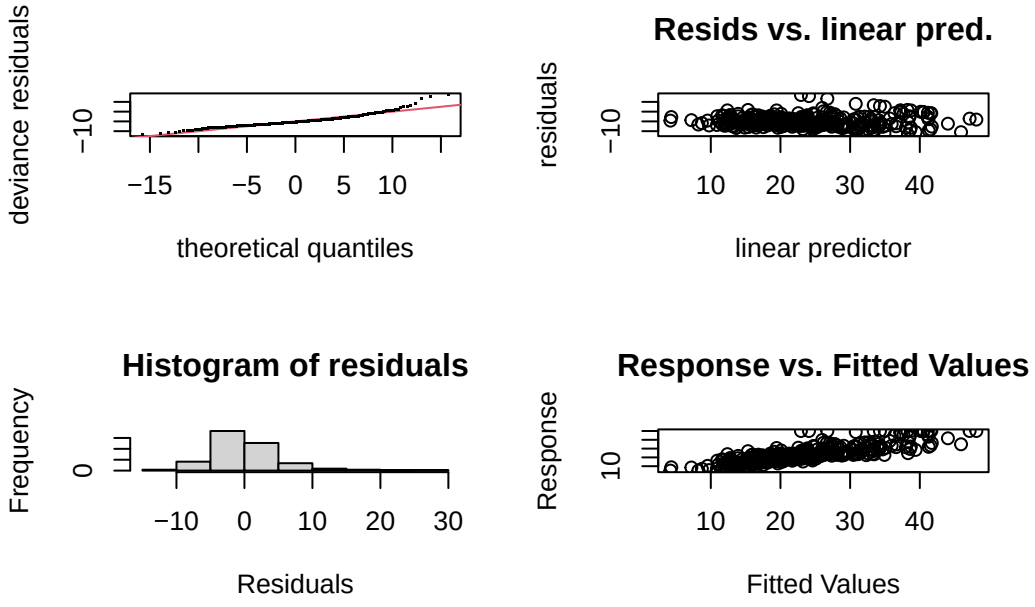
To investigate, models with lower `k` values of 3, 4, and 5 were tested, all of which produced edf values close to 1 and very similar AIC scores, indicating that the relationship between `nox` and `medv` is closer to linear than nonlinear. Given this evidence, `nox` was fit as a linear term in the full GAM, which is both more interpretable and better supported by the data.

Fit final model with `nox` as a linear term and splines for the other two

```
# Refit full GAM with nox as linear term
gam_fit <- gam(medv ~ s(crim) + nox + s(lstat),
               data = train,
               method = "REML")

summary(gam_fit)
```

```
Family: gaussian
Link function: identity

Formula:
medv ~ s(crim) + nox + s(lstat)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   19.039      1.654  11.509   <2e-16 ***
nox            6.909      2.954   2.339   0.0199 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
           edf Ref.df     F  p-value
s(crim)  1.002  1.004 15.96 7.67e-05 ***
s(lstat) 6.675  7.802 81.40  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.699   Deviance explained = 70.6%
-REML = 1245.8  Scale est. = 27.154    n = 404
```

```
gam.check(gam_fit)
```



```
Method: REML   Optimizer: outer newton
full convergence after 6 iterations.
Gradient range [-0.0004381934,0.0004832646]
(score 1245.842 & scale 27.15395).
Hessian positive definite, eigenvalue range [0.0004381404,200.0402].
Model rank =  20 / 20

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

           k'  edf k-index p-value
s(crim)  9.00 1.00    1.07    0.93
s(lstat) 9.00 6.68    1.06    0.85
```

Evaluation

```
gam_preds <- predict(gam_fit, newdata = test)
gam_test_rmse <- sqrt(mean((test$medv - gam_preds)^2))
cat("GAM Test RMSE:", gam_test_rmse)
```

```
GAM Test RMSE: 4.801678
```

Both smooth terms were highly significant, with `lstat` showing a strong nonlinear relationship with `medv` at an edf of 6.68, while `crim` behaved nearly linearly at an edf of 1.00. The model explained 70% of the variation in `medv` and achieved a test RMSE of 4.80, meaning predictions of median home price were off by about $4,800 on average.

Random Forests

```
library(randomForest)
```

```
randomForest 4.7-1.2

Type rfNews() to see new features/changes/bug fixes.
```

```r
# Initial random forest with default parameters
set.seed(1)
rf_fit_initial <- randomForest(medv ~ crim + nox + lstat,
                               data = train,
                               importance = TRUE)
rf_fit_initial
```

```
Call:
 randomForest(formula = medv ~ crim + nox + lstat, data = train,      importance = TRUE)
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 1

          Mean of squared residuals: 20.55731
                    % Var explained: 77.16
```

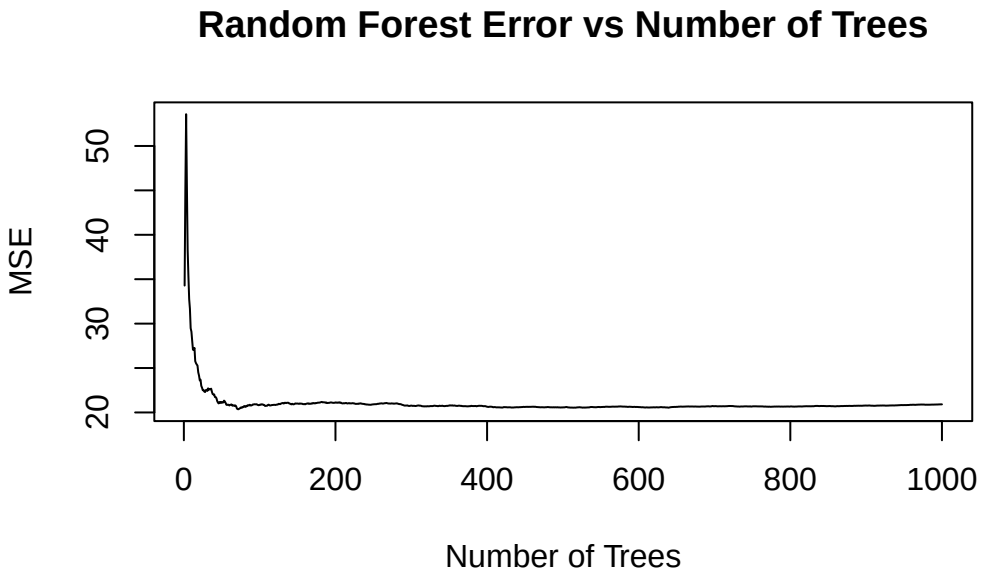
```r
set.seed(1)
mtry_values <- c(1, 2, 3)
mtry_errors <- rep(NA, length(mtry_values))

for (i in seq_along(mtry_values)) {
  rf_fit <- randomForest(medv ~ crim + nox + lstat,
                         data = train,
                         mtry = mtry_values[i],
                         ntree = 500)
  mtry_errors[i] <- mean(rf_fit$mse)
}

best_mtry <- mtry_values[which.min(mtry_errors)]
cat("Best mtry:", best_mtry)
```

```
Best mtry: 1
```

Only one predictor will be considered at each split which makes sense considering we are only evaluating based on three predictors.

```r
set.seed(1)
rf_fit_ntree <- randomForest(medv ~ crim + nox + lstat,
                             data = train,
```

```
                              mtry = best_mtry,
                              ntree = 1000)

# Plot error curve to see where it stabilizes
plot(rf_fit_ntree$mse, type = "l",
     xlab = "Number of Trees",
     ylab = "MSE",
     main = "Random Forest Error vs Number of Trees")
```



The error curve flattened out around 200 trees, so `ntree = 200` was chosen as the point where adding more trees no longer improved performance.

```
set.seed(1)
for (ns in c(1, 3, 5)) {
  rf_fit <- randomForest(medv ~ crim + nox + lstat,
                         data = train,
                         mtry = 1,
                         ntree = 200,
                         nodesize = ns)
  preds <- predict(rf_fit, newdata = test)
  error <- mean((test$medv - preds)^2)
  cat("Nodesize:", ns, "| Test MSE:", error, "\n")
}
```

```
Nodesize: 1 | Test MSE: 12.2136
```

```
Nodesize: 3 | Test MSE: 12.06182
Nodesize: 5 | Test MSE: 12.05451
```

The test MSE was lowest at `nodesize = 5` (12.05) compared to `nodesize = 3` (12.06) and `nodesize = 1` (12.21), so `nodesize = 5` was selected as the final value. While the differences are small, this choice also prevents the trees from growing unnecessarily deep, reducing the risk of overfitting to the training data.

Final Random Forest Model

```r
set.seed(1)
rf_fit_final <- randomForest(medv ~ crim + nox + lstat,
                             data = train,
                             mtry = 1,
                             ntree = 200,
                             nodesize = 5,
                             importance = TRUE)

rf_fit_final
```

```
Call:
 randomForest(formula = medv ~ crim + nox + lstat, data = train,      mtry = 1, ntree = 200,
               Type of random forest: regression
                     Number of trees: 200
No. of variables tried at each split: 1

          Mean of squared residuals: 20.63281
                    % Var explained: 77.08
```

Evaluation

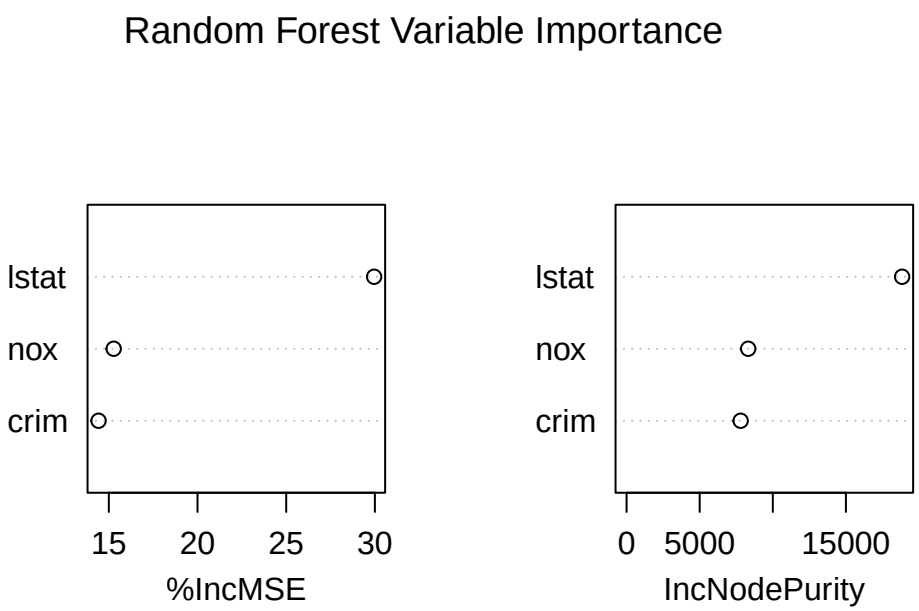
```r
# Test RMSE
rf_preds <- predict(rf_fit_final, newdata = test)
rf_test_rmse <- sqrt(mean((test$medv - rf_preds)^2))
cat("Random Forest Test RMSE:", rf_test_rmse)
```

```
Random Forest Test RMSE: 3.462082
```

```r
# Variable Importance
importance(rf_fit_final)
```

```
       %IncMSE IncNodePurity
crim  14.42163      7801.345
nox   15.27418      8317.568
lstat 29.95677     18844.074
```

```r
varImpPlot(rf_fit_final, main = "Random Forest Variable Importance")
```



The Random Forest model achieved a test RMSE of 3.46, meaning predictions of median home price are off by about $3,460 on average. Both the %IncMSE and IncNodePurity metrics agree that `lstat` is by far the most important predictor, with `nox` and `crim` contributing considerably less to the model's predictions. This aligns with our earlier EDA which showed `lstat` had the clearest and most consistent negative relationship with `medv` of the three predictors.

PCR Model

```r
library(pls)
```

```
Attaching package: 'pls'
```

```
The following object is masked from 'package:stats':

    loadings
```

```r
set.seed(1)
pcr_fit <- pcr(medv ~ crim + nox + lstat,
               data = train,
               scale = TRUE,
               validation = "CV")
summary(pcr_fit)
```

```
Data:   X dimension: 404 3
    Y dimension: 404 1
Fit method: svdpc
Number of components considered: 3

VALIDATION: RMSEP
Cross-validated using 10 random segments.
       (Intercept)  1 comps  2 comps  3 comps
CV           9.511    7.355    7.245    6.429
adjCV        9.511    7.351    7.241    6.426

TRAINING: % variance explained
      1 comps  2 comps  3 comps
X       66.07    86.05   100.00
medv    40.68    42.18    54.99
```

Based on these results we will use 3 components.

Evaluation

```r
pcr_preds <- predict(pcr_fit, newdata = test, ncomp = 3)
pcr_test_rmse <- sqrt(mean((test$medv - pcr_preds)^2))
cat("PCR Test RMSE:", pcr_test_rmse)
```

```
PCR Test RMSE: 5.414879
```

PCR was fit using 10-fold cross-validation, with all 3 components selected since each one improved prediction accuracy, together explaining 55% of the variation in `medv`. The test RMSE of 5.41 was much higher than Random Forest's 3.46, which makes sense given that PCR assumes linear relationships and our data showed clear nonlinear patterns earlier.

Conclusion

```
# Compare all model RMSEs
results <- data.frame(
  Model = c("PCR", "GAM", "Random Forest"),
  RMSE = c(pcr_test_rmse, gam_test_rmse, rf_test_rmse)
)

results[order(results$RMSE), ]
```

```
          Model     RMSE
3 Random Forest 3.462082
2           GAM 4.801678
1           PCR 5.414879
```

Random Forest was the clear winner out of these models, with a test RMSE of 3.46, outperforming GAM at 4.80 and PCR at 5.41. PCR struggled most as it assumes linear relationships which our EDA showed were not appropriate for this data, GAM improved on this by capturing nonlinearity in `crim` and `lstat`, and Random Forest performed best by making no assumptions about the functional form of any relationship and naturally handling the complexity in the data.